COSMETICS MANAGEMENT SYSTEM

ABSTRACT

The present generation is highly influenced by technology, which has transformed various sectors, including the beauty and cosmetics industry. The cosmetics sector plays a crucial role in both online and offline retail, allowing customers to purchase products conveniently. Many customers prefer pre-ordering their favorite beauty products to avoid last-minute unavailability.

Earlier, customers had to visit physical stores or rely on local beauty shops for purchasing cosmetics, which was not always accessible. This created difficulties in checking product availability and comparing options before making a purchase. With the increasing digital awareness of today's mobile generation, this issue can be resolved using a simple **online web application** that displays all the details of a cosmetics management system, including product availability, pricing, and customer reviews.

To manage this efficiently, a **database** is required to store and organize information systematically. This database will keep records of customers, beauty products, orders, inventory, transactions, and customer feedback. The database will also support dynamic operations such as adding new products, updating prices, tracking stock levels, processing orders, and managing returns.

Key Features of the Cosmetics Management System:

- **Product Catalog**: Stores details of different cosmetic products, including categories like skincare, makeup, hair care, and fragrances.
- **Customer Management**: Maintains customer details, including purchase history, preferences, and feedback.
- **Order Processing**: Manages orders, payments, shipping, and cancellations.
- **Inventory Tracking**: Monitors stock levels and updates availability status.
- **Admin Panel**: Allows administrators to add, modify, or remove products and manage users.
- **Feedback & Reviews**: Enables customers to share reviews and ratings for products.

Technologies Used:

- **Database Management**: SQL Server, Microsoft SQL Server Management Studio (SSMS)
- **Frontend Technologies**: HTML, CSS, JavaScript
- **Backend Technologies**: PHP Server

TABLE OF CONTENTS

Title	Page No.
ABSTRACT	i
CHAPTER 1. INTRODUCTION	
1.1 Introduction to the topic	1-2
1.2 Problem Statement	1-2
1.3 Objectives	2-2
CHAPTER 2. DATABASE DESIGN	
2.1 List of Attributes, entities and relationship	4-6
2.2 E-R Diagram	7-8
CHAPTER 3. RELATIONAL MODEL	
3.1 Database languages	9-10
3.2 Table Description	10-11
3.3 Relational Database Scheme	12-17
3.4 Relational Queries	18-42
CHAPTER 4. CONCLUSION AND FUTUREWORK	
4.1 Conclusion	43-43
4.2 Future Work	43-43

CHAPTER 1. INTRODUCTION

1.1 Introduction to the Topic

A **Cosmetics Management System** is an online platform that helps manage beauty and skincare products. Before such systems, cosmetic store owners had to manually track inventory, manage sales, and record customer details using spreadsheets or paper records.

Now, a **Cosmetics Management System** makes things easier by updating stock, processing orders, handling payments, and keeping customer information automatically. It also helps customers by letting them browse products, check reviews, and make purchases in one place without needing to visit different websites.

This system also provides useful features like generating sales reports, tracking customer preferences, and ensuring that stock is well-managed to avoid running out of products or having too many in storage.

1.2 Problem Statement:

This website is a **dynamic shopping platform** where users can explore and buy cosmetic products like skincare, makeup, and haircare. The system manages different types of information, including product details, customer accounts, stock levels, order history, payment processing, and customer reviews.

To use the website, customers first need to sign up by providing basic details like their name, email, and delivery address. Once registered, they get a login ID and password. After logging in, users can search for beauty products, filter them by category or brand, and add items to their shopping cart.

When a customer places an order, the system checks if the products are available and calculates the total price, including taxes and delivery charges. Customers can pay using credit/debit cards, UPI, net banking, or cash on delivery. After successful payment, the system generates an invoice and confirms the order. If a product is out of stock, it suggests similar alternatives.

The website is designed to be **simple and user-friendly**, making it easy to use for people of all ages. Its smooth navigation helps customers shop quickly and efficiently.

1.3 Objectives

The **Cosmetics Management System** helps manage beauty products efficiently and improves customer experience. Key objectives include:

- **Product Management:** List and organize beauty products like skincare, makeup, and haircare with price, brand, and stock details.
- **Customer Management:** Keep customer records, track order history, and provide recommendations based on past purchases.

- **Order Processing:** Handle orders, payments, and invoices smoothly for easy transactions.
- **Inventory Management:** Keep track of stock levels and notify admins when products are running low.
- **Admin Panel:** Allow admins to update product listings, respond to customer inquiries, track sales, and manage orders.
- **Payment Integration:** Support different payment methods to make transactions secure and easy.
- **Feedback and Reviews:** Let customers share their experiences and rate products to help others make better choices.

This system makes it easier for cosmetic businesses to manage their products while giving customers a smooth and enjoyable shopping experience. It ensures efficiency, accuracy, and customer satisfaction.

CHAPTER 2. DATABASE DESIGN

2.1 List of Attributes, entities and relationship

1. **Entity Name: Customer**

Attributes	Type
Customer id	int(10)
Name	varchar(255)
Email	varchar(255)
Phone	varchar(20)
Address	text

2. **Entity Name: Product**

Attributes	Type
Productid	int(10)
Name	varchar(255)
Brand id	int(10)
Category id	int(10)
price	decimal(10,2)
Stock Quantity	int(10)
Expiry Date	Int(10)

3. **Entity Name: Category**

Attributes	Type
Category id	int(10)
Category Name	varchar(255)
Description	text

4. **Entity Name: Brand**

Attributes	Type
Brand id	int(10)
Brand Name	Varchar(255)
Country	varchar(100)
Description	text

5. **Entity Name: Order**

Attributes	Type
Order id	int(10)
Customer id	int(10)
Order Date	date
Total Amount	Decimal(10,2)
Status	varchar(50)

6. **Entity Name: Order Details**

Attributes	Type
Order Details	int(10)
Order id	Int(10)
Product id	Int(10)
Quantity	Int(10)
Sub Total	Decimal(10,2)

7. **Entity Name: Supplier**

Attributes	Type
Supplier id	int(10)
Supplier Name	Varchar(255)
Contact Name	Varchar(255)
Phone	Varchar(20)
Email	Varchar(255)
Address	text

8. **Entity Name: Purchase**

Attributes	Type
Purchase id	int(10)
Supplier id	int(10)
Purchase Date	date
Total Cost	Decimal(10,2)

9. **Entity Name: Purchase Details**

Attributes	Type
Purchase Details id	int(10)
Purchase id	int(10)
Product id	Int(10)
Quantity	Int(10)
Cost price	Decimal(10,2)

10. **Entity Name: Inventory**

Attributes	Type
Inventory id	int(10)
Product id	int(10)
Stock Level	Int(10)
Reorder Level	Int(10)
Last Updated	date

11. **Entity Name: Employee**

Attributes	Type
Employee id	int(10)
Name	varchar(255)
Email	varchar(255)
Phone	varchar(20)

Role	varchar(100)
Salary	Decimal(10,2)

12. **Entity Name: Sales**

Attributes	**Type**
Sales id	int(10)
Order id	Int(10)
Sales Date	date
Total Revenue	Decimal(10,2)

13. **Entity Name: Payment**

Attributes	**Type**
Payment id	int(10)
Order id	Int(10)
Payment Status	varchar(50)
Payment Method	varchar(50)
Transaction id	varchar(255)

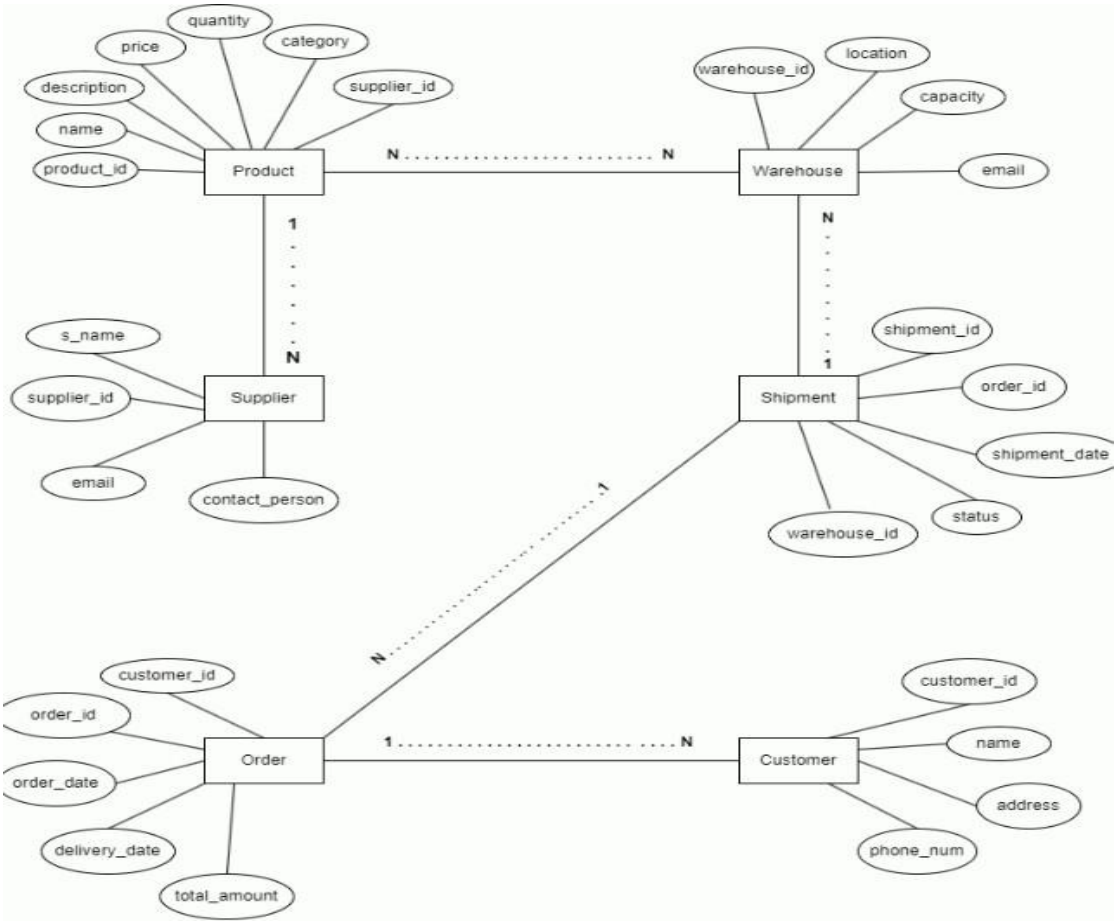
14. **Entity Name: Review**

Attributes	**Type**
Review id	int(10)
Customer id	Int(10)
Product id	Int(10)
Rating	Int(10)
Comment	Text

15. **Entity Name: Discount**

Attributes	**Type**
Discount id	int(10)
Discount Name	Varchar(255)
Discount Percentage	Decimal(5,2)
Start Date	date
End Date	date

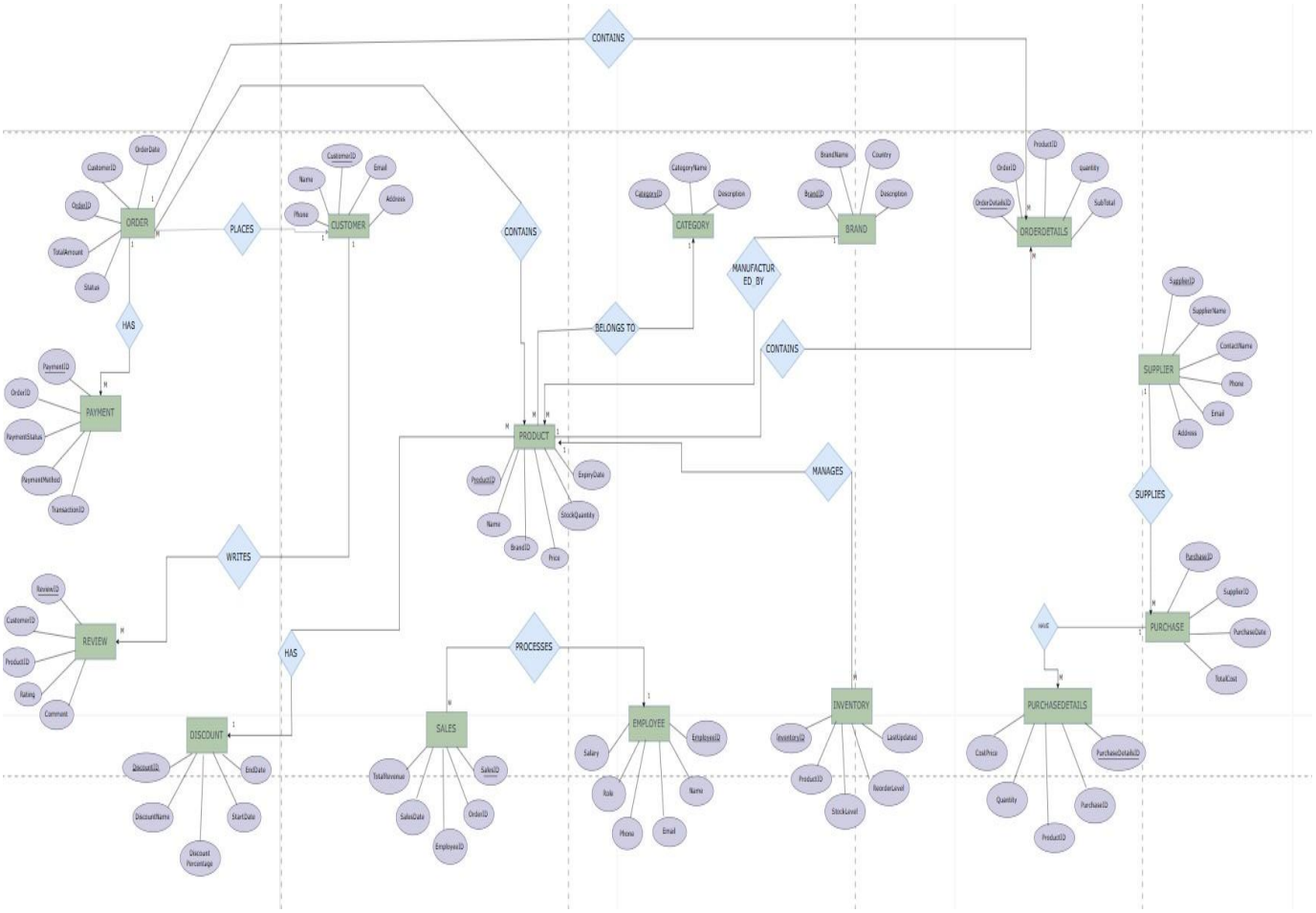
2.1.1 Entities and their relationships:



The above diagram is a simple representation of entities which shows the connectivity between all the entities and the relationship between various entities

To know in detail about the types of relationships that exist between all the entities and to know the different attributes that describes about the entity we design ER(entity relation) diagram.

2.2 E-R Diagram


CONTAINS
ORDER
OrderDate
CustomerID
OrderID
TotalAmount
Status
PLACES
CUSTOMER
CustomerID
Name
Email
Phone
Address
CONTAINS
CATEGORY
CategoryName
CategoryID
Description
BRAND
BrandName
Country
BrandID
Description
ORDERDETAILS
ProductID
OrderID
quantity
OrderDetailsID
SubTotal
HAS
PAYMENT
PaymentID
OrderID
PaymentStatus
PaymentMethod
TransactionID
MANUFACTURED_BY
BELONGS TO
CONTAINS
SUPPLIER
SupplierID
SupplierName
ContactName
Phone
Email
Address
PRODUCT
ProductID
Name
BrandID
Price
StockQuantity
ExpiryDate
MANAGES
SUPPLIES
WRITES
REVIEW
ReviewID
CustomerID
ProductID
Rating
Comment
HAS
PROCESSES
PURCHASE
PurchaseID
SupplierID
PurchaseDate
TotalCost
HAVE
DISCOUNT
DiscountID
DiscountName
Discount Percentage
StartDate
EndDate
SALES
TotalRevenue
SalesDate
EmployeeID
OrderID
SalesID
EMPLOYEE
Salary
Role
Phone
Email
Name
EmployeeID
INVENTORY
InventoryID
ProductID
StockLevel
ReorderLevel
LastUpdated
PURCHASEDETAILS
CostPrice
Quantity
ProductID
PurchaseID
PurchaseDetailsID

CHAPTER 3. RELATIONAL MODEL

3.1 Database languages

Four categories of database languages :

1. Data definition language (DDL)

Data definition language (DDL) creates the framework of the database by specifying the database schema, which is the structure that represents the organization of data. Its common uses include the creation and alteration of tables, files, indexes and columns within the database. This language also allows users to rename or drop the existing database or its components.

Here's a list of DDL statements:

- CREATE: Creates a new database or object, such as a table, index or column.
- ALTER: Changes the structure of the database or object.
- DROP: Deletes the database or existing objects.
- RENAME: Renames the database or existing objects.

2. Data manipulation language (DML)

Data manipulation language (DML) provides operations that handle user requests, offering a way to access and manipulate the data that users store within a database. Its common functions include inserting, updating and retrieving data from the database.

Here's a list of DML statements:

- INSERT: Adds new data to the existing database table.
- UPDATE: Changes or updates values in the table.
- DELETE: Removes records or rows from the table.
- SELECT: Retrieves data from the table or multiple tables.

3. Data control language (DCL)

Data control language (DCL) controls access to the data that users store within a database. Essentially, this language controls the rights and permissions of the database system. It allows users to grant or revoke privileges to the database.

Here's a list of DCL statements:

- GRANT: Gives a user access to the database.
- REVOKE: Removes a user's access to the database.

4. Transaction control language (TCL)

Transaction control language (TCL) manages the transactions within a database. Transactions group a set of related tasks into a single, executable task. All the tasks must succeed in order for the transaction to work. Here's a list of TCL statements:

- COMMIT: Carries out a transaction.

- ROLLBACK: Restores a transaction if any tasks fail to execute.

3.2 Table Description :

Following are the tables along with constraints used in Cosmetics Management System Database.

1. **Customer**: This table stores customer details like ID, name, email, phone, and address.
    - Constraint: customer id is the primary key and uniquely identifies each customer.
2. **Product**: This table contains product details such as name, brand, category, price, stock quantity, and expiry date.
    - Constraints: product id is the primary key. Brand id and category id are foreign keys linking to the Brand and Category tables.
3. **Category**: Stores product categories like skincare, makeup, or haircare.
    - Constraint: category id is the primary key.
4. **Brand**: Holds brand information, including name, country, and description.
    - Constraint: brand id is the primary key.
5. **Order**: Contains customer orders with order date, total amount, and status.
    - Constraints: order id is the primary key. Customer id is a foreign key linking to the Customer table.
6. **Order Detail**: A bridge table connecting orders and products, tracking quantity and subtotal per product.
    - Constraints: order detail id is the primary key. Order id and product id are foreign keys.
7. **Supplier**: Stores supplier details like name, contact person, phone, email, and address.
    - Constraint: supplier id is the primary key.
8. **Purchase**: Manages purchases made from suppliers, including purchase date and total cost.
    - Constraints: purchase id is the primary key. Supplier id is a foreign key.
9. **Purchase Detail**: A bridge table for purchases and products, storing quantity and cost price.
    - Constraints: purchase detail id is the primary key. Purchase id and product id are foreign keys.
10. **Inventory**: Keeps track of stock levels and reorder levels.
    - Constraints: inventory id is the primary key. Product id is a foreign key.
11. **Employee**: Contains employee details such as name, role, salary, and contact information.
    - Constraint: employee id is the primary key.
12. **Sales**: Tracks sales transactions, linking employees to processed orders.
    - Constraints: sales id is the primary key. Order id and employee id are foreign keys.
13. **Payment**: Stores payment details like method, status, and transaction ID.
    - Constraints: payment id is the primary key. Order id is a foreign key.
14. **Review**: Captures customer feedback on products with ratings and comments.
    - Constraints: review id is the primary key. Customer id and product id are foreign keys.

15. **Discount**: Defines discount offers, including name, percentage, start and end dates.
    - Constraint: discount id is the primary key.

The above descriptions explain the database structure, entities, attributes, and relationships for managing a cosmetics store efficiently.

3.3 Relational Database Scheme

The relational database schema for ***Cosmetics Management System*** database is as follows:

1. Customer(customer id, name, email, phone, address)
2. Product(productid, name, brand id, category id, price, stock quantity, expiry date)
3. Category(category id, category name, description)
4. Brand(brand id, brand name, country, description)
5. Order(order id, customer id, order date, total amount, status)
6. Order Detail(order detail id, order id, product id, quantity, subtotal)
7. Supplier(supplier id, supplier name, contact name, phone, email, address)
8. Purchase(purchase id, supplier id, purchase date, total cost)
9. Purchase Detail(purchase detail id, purchase id, product id, quantity, cost price)
10. Inventory(inventory id, product id, stock level, reorder level, last updated)
11. Employee(employee id, name, phone, email, role, salary)
12. Sales(sales id, order id, employee id, sales date, total revenue)
13. Payment(payment id, order id, payment method, payment status, transaction id)
14. Review(review id, customer id, product id, rating, comment, review date)
15. Discount(discount id, discount name, discount percentage, start date, end date)

3.4 Relational Queries:

```
CREATE TABLE CUSTOMER (
    CustomerID INT PRIMARY KEY,
    Name VARCHAR(255),
    Email VARCHAR(255),
    Phone VARCHAR(20),
    Address TEXT
);

CREATE TABLE ORDER_TABLE (
    OrderID INT PRIMARY KEY,
    CustomerID INT,
    OrderDate DATE,
    TotalAmount DECIMAL(10,2),
    Status VARCHAR(50),
    FOREIGN KEY (CustomerID) REFERENCES CUSTOMER(CustomerID)
);

CREATE TABLE PAYMENT (
    PaymentID INT PRIMARY KEY,
    OrderID INT,
    PaymentStatus VARCHAR(50),
    PaymentMethod VARCHAR(50),
    TransactionID VARCHAR(255),
    FOREIGN KEY (OrderID) REFERENCES ORDER_TABLE(OrderID)
);

CREATE TABLE REVIEW (
    ReviewID INT PRIMARY KEY,
    CustomerID INT,
    ProductID INT,
    Rating INT,
    Comment TEXT,
    FOREIGN KEY (CustomerID) REFERENCES CUSTOMER(CustomerID)
);

CREATE TABLE DISCOUNT (
```

```
    DiscountID INT PRIMARY KEY,
    DiscountName VARCHAR(255),
    DiscountPercentage DECIMAL(5,2),
    StartDate DATE,
    EndDate DATE
);

CREATE TABLE CATEGORY (
    CategoryID INT PRIMARY KEY,
    CategoryName VARCHAR(255),
    Description TEXT
);

CREATE TABLE BRAND (
    BrandID INT PRIMARY KEY,
    BrandName VARCHAR(255),
    Country VARCHAR(100),
    Description TEXT
);

CREATE TABLE PRODUCT (
    ProductID INT PRIMARY KEY,
    Name VARCHAR(255),
    BrandID INT,
    CategoryID INT,
    Price DECIMAL(10,2),
    StockQuantity INT,
    ExpiryDate DATE,
    FOREIGN KEY (BrandID) REFERENCES BRAND(BrandID),
    FOREIGN KEY (CategoryID) REFERENCES CATEGORY(CategoryID)
);

CREATE TABLE ORDERDETAILS (
    OrderDetailsID INT PRIMARY KEY,
    OrderID INT,
    ProductID INT,
    Quantity INT,
    SubTotal DECIMAL(10,2),
    FOREIGN KEY (OrderID) REFERENCES ORDER_TABLE(OrderID),
```

```
    FOREIGN KEY (ProductID) REFERENCES PRODUCT(ProductID)
);

CREATE TABLE SUPPLIER (
    SupplierID INT PRIMARY KEY,
    SupplierName VARCHAR(255),
    ContactName VARCHAR(255),
    Phone VARCHAR(20),
    Email VARCHAR(255),
    Address TEXT
);

CREATE TABLE PURCHASE (
    PurchaseID INT PRIMARY KEY,
    SupplierID INT,
    PurchaseDate DATE,
    TotalCost DECIMAL(10,2),
    FOREIGN KEY (SupplierID) REFERENCES SUPPLIER(SupplierID)
);

CREATE TABLE PURCHASEDETAILS (
    PurchaseDetailsID INT PRIMARY KEY,
    PurchaseID INT,
    ProductID INT,
    Quantity INT,
    CostPrice DECIMAL(10,2),
    FOREIGN KEY (PurchaseID) REFERENCES PURCHASE(PurchaseID),
    FOREIGN KEY (ProductID) REFERENCES PRODUCT(ProductID)
);

CREATE TABLE SALES (
    SalesID INT PRIMARY KEY,
    OrderID INT,
    SalesDate DATE,
    TotalRevenue DECIMAL(10,2),
    FOREIGN KEY (OrderID) REFERENCES ORDER_TABLE(OrderID)
);

CREATE TABLE EMPLOYEE (
```

```
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(255),
    Email VARCHAR(255),
    Phone VARCHAR(20),
    Role VARCHAR(100),
    Salary DECIMAL(10,2)
);

CREATE TABLE INVENTORY (
    InventoryID INT PRIMARY KEY,
    ProductID INT,
    StockLevel INT,
    ReorderLevel INT,
    LastUpdated DATE,
    FOREIGN KEY (ProductID) REFERENCES PRODUCT(ProductID)
);

INSERT INTO CUSTOMER (CustomerID, Name, Email, Phone, Address) VALUES
(1, 'Ravi', 'ravi313@gmail.com', '1234567890', 'Tirupati'),
(2, 'Rani', 'rani2766@gmail.com', '2345678901', 'Tirupati'),

INSERT INTO ORDER_TABLE (OrderID, CustomerID, OrderDate, TotalAmount, Status) VALUES
(21, 1, '2025-03-01', 150.50, 'Completed'),
(22, 2, '2025-03-02', 200.00, 'Pending'),

INSERT INTO PAYMENT (PaymentID, OrderID, PaymentStatus, PaymentMethod, TransactionID)
VALUES
(41, 21, 'Success', 'Credit Card', 'TXN1001'),
(42, 22, 'Pending', 'PayPal', 'TXN1002'),

INSERT INTO REVIEW (ReviewID, CustomerID, ProductID, Rating, Comment) VALUES
(61, 1, 81, 5, 'Amazing lipstick, long-lasting!'),
(62, 2, 82, 4, 'Great foundation, but a little dry for my skin'),

INSERT INTO DISCOUNT (DiscountID, DiscountName, DiscountPercentage, StartDate, EndDate)
VALUES
(101, 'Spring Beauty Sale', 10.00, '2025-03-01', '2025-03-10'),
(102, 'Summer Glow Deal', 15.00, '2025-06-01', '2025-06-15'),
```

INSERT INTO CATEGORY (CategoryID, CategoryName, Description) VALUES
(121, 'Lipsticks', 'Matte, glossy, and liquid lipsticks'),
(122, 'Foundations', 'Full coverage, lightweight, and skin tints'),

INSERT INTO BRAND (BrandID, BrandName, Country, Description) VALUES
(141, 'Maybelline', 'USA', 'Affordable and trendy cosmetics'),
(142, 'MAC', 'Canada', 'High-end professional makeup brand'),

INSERT INTO PRODUCT (ProductID, Name, BrandID, CategoryID, Price, StockQuantity, ExpiryDate) VALUES
(81, 'SuperStay Matte Ink Lipstick', 141, 121, 9.99, 150, '2026-06-01'),
(82, 'Studio Fix Fluid Foundation', 142, 122, 34.99, 100, '2026-08-15'),

INSERT INTO ORDERDETAILS (OrderDetailsID, OrderID, ProductID, Quantity, SubTotal) VALUES
(161, 21, 81, 2, 19.98),
(162, 22, 82, 1, 34.99),

INSERT INTO SUPPLIER (SupplierID, SupplierName, ContactName, Phone, Email, Address) VALUES
(181, 'Cosmetic Suppliers Inc.', 'mahesh', '1234567890', 'mahesh23@cosmeticsuppliers.com', 'andhra pradesh'),
(182, 'Glamour Distributors', 'indu', '9876543210', 'indu@glamourdist.com', 'chennai'),

INSERT INTO PURCHASE (PurchaseID, SupplierID, PurchaseDate, TotalCost) VALUES
(201, 181, '2025-01-15', 1200.50),
(202, 182, '2025-02-10', 850.75),

INSERT INTO PURCHASEDETAILS (PurchaseDetailsID, PurchaseID, ProductID, Quantity, CostPrice) VALUES
(221, 201, 81, 50, 8.50),
(222, 202, 82, 30, 28.00),
INSERT INTO SALES (SalesID, OrderID, SalesDate, TotalRevenue) VALUES
(241, 21, '2024-03-01', 250.00),
(242, 22, '2024-03-02', 150.50),

INSERT INTO EMPLOYEE (EmployeeID, Name, Email, Phone, Role, Salary) VALUES
(261, 'rajesh', 'rajesh974@cosmetics.com', '1112223333', 'Store Manager', 55000.00),
(262, 'leena', 'leena37@cosmetics.com', '2223334444', 'Sales Associate', 32000.00),

```
INSERT INTO INVENTORY (InventoryID, ProductID, StockLevel, ReorderLevel, LastUpdated)
VALUES
(281, 81, 150, 50, '2024-03-01'),
(282, 82, 120, 30, '2024-03-02'),

select * from customer;
select * from order_table;

select * from payment;

select * from review;

select * from discount;

select * from category;

select * from brand;

select * from product;

select * from orderdetails;

select * from supplier;

select * from purchase;

select * from purchasedetails;

select * from sales;

select * from employee;

select * from inventory;
```

SQL QUERIES:

-- Query1. Select customers with IDs greater than 10

SELECT * FROM CUSTOMER

WHERE CustomerID > 10;

OUTPUT:

/*11 teja teja34@gmail.com 1111111111 hyderabad

12 pavani pavani267@gmail.com 2222222222 chittoor

13 thanuja thanuja135@gmail.com 3333333333 kadiri

14 vinitha vinitha4363@gmail.com 4444444444 nellore

15 raji raji245@gmail.com 5555555555 kurnool

16 santhosh santhosh234@gmail.com 6666666666 bangalore

17 prakash prakash296@gmail.com7777777777 mysore

18 Roshini roshini34@gmail.com 8888888888 chennai

19 meghana meghana4868@gmail.com 9999999999 amaravathi

20 nandini nandini245@gmail.com 0000000000 tirupati*/

--Query 2. Select payments with status 'Success'

SELECT * FROM PAYMENT

WHERE PaymentStatus = 'Success';

OUTPUT:

/*41 21 Success Credit Card TXN1001

43 23 Success Debit Card TXN1003

45 25 Success PayPal TXN1005

46 26 Success Credit Card TXN1006

48 28 Success PayPal TXN1008

49 29 Success Credit Card TXN1009

50 30 Success Debit Card TXN1010

52 32 Success Credit Card TXN1012

53 33 Success Debit Card TXN1013

55 35 Success Credit Card TXN1015

56 36 Success Debit Card TXN1016

58 38 Success PayPal TXN1018

59 39 Success Debit Card TXN1019

60 40 Success Credit Card TXN1020*/

-- Query3. Select discounts with percentage greater than 20

SELECT * FROM DISCOUNT

WHERE DiscountPercentage > 20;

OUTPUT:

/*104 Black Friday Beauty 30.00 2025-11-25 2025-11-30
105 New Year Makeup Offer 25.00 2026-01-01 2026-01-05
106 Clearance Cosmetics 40.00 2025-09-15 2025-09-20
107 Cyber Monday Skincare 35.00 2025-11-28 2025-11-29
114 Independence Day Makeup Sale 22.00 2025-07-04 2025-07-05
116 Mega Beauty Sale 50.00 2025-10-01 2025-10-07
117 Flash Skincare Sale 33.00 2025-06-15 2025-06-16
118 Diwali Makeup Offer 27.00 2025-10-20 2025-10-25*/

-- Query4. Select order details with quantity greater than 2

SELECT * FROM ORDERDETAILS
WHERE Quantity > 2;

OUTPUT:

/*163 23 83 3 32.97
167 27 87 4 31.96
175 35 95 3 71.97*/

--Query5. Select purchases with total cost greater than 1500

SELECT * FROM PURCHASE
WHERE TotalCost > 1500;

OUTPUT:

/*213 193 2025-11-20 1550.40
218 198 2025-12-15 1510.65
220 200 2025-12-25 1600.45*/

--Query 6. Select payments made by 'Credit Card'

SELECT * FROM PAYMENT
WHERE PaymentMethod = 'Credit Card';

OUTPUT:

/*41 21 Success Credit Card TXN1001
44 24 Failed Credit Card TXN1004
46 26 Success Credit Card TXN1006
49 29 Success Credit Card TXN1009
52 32 Success Credit Card TXN1012
55 35 Success Credit Card TXN1015
57 37 Failed Credit Card TXN1017
60 40 Success Credit Card TXN1020*/

-- Query7. Select discounts that start in '2025-03'

```
SELECT * FROM DISCOUNT
WHERE StartDate LIKE '2025-03%';
```

OUTPUT:

```
/*101    Spring Beauty Sale    10.00    2025-03-01    2025-03-10*/
```

-- Query8. Select employees with role 'Sales Associate'

```
SELECT * FROM EMPLOYEE
WHERE Role = 'Sales Associate';
```

OUTPUT:

```
/*262    leena    leena37@cosmetics.com        2223334444    Sales Associate 32000.00
266      vasu     vasu876@cosmetics.com        6667778888    Sales Associate 31000.00
272      rohan    rohan976@cosmetics.com       2223334445    Sales Associate 31500.00*/
```

--Query 9. Select inventory with stock level less than 100

```
SELECT * FROM INVENTORY
WHERE StockLevel < 100;
```

OUTPUT:

```
/*284    84    80    25    2024-03-04
287      87    90    30    2024-03-07
290      90    95    25    2024-03-10
294      94    60    20    2024-03-14
300      100   85    30    2024-03-20*/
```

-- Query10. Count the number of products

```
SELECT COUNT(*)
FROM PRODUCT;
```

OUTPUT:

```
/*20*/
```

-- Query11. Calculate the average product price

```
SELECT AVG(Price)
FROM PRODUCT;
```

OUTPUT:

```
/*26.840000*/
```

-- Query12. Calculate the total cost of purchases

```
SELECT SUM(TotalCost)
FROM PURCHASE;
```

OUTPUT:

/*25206.05*/

-- Query13. Find the minimum product price

```
SELECT MIN(Price)
FROM PRODUCT;
```

OUTPUT:

/*5.99*/

-- Query14. Find the maximum salary of employees

```
SELECT MAX(Salary)
FROM EMPLOYEE;
```

OUTPUT:

/*60000.00*/

-- Query15. Group products by brand and count them

```
SELECT BrandID, COUNT(*)
FROM PRODUCT
GROUP BY BrandID;
```

OUTPUT:

/*141 1
142 1
143 1
144 1
145 1
146 1
147 1
148 1
149 1
150 1
151 1
152 1
153 1
154 1
155 1
156 1

157 1
158 1
159 1
160 1*/

-- Query16. Join CUSTOMER and ORDER_TABLE tables

```
SELECT * FROM CUSTOMER
JOIN ORDER_TABLE ON CUSTOMER.CustomerID = ORDER_TABLE.CustomerID;
```

OUTPUT:

/*1 Ravi ravi313@gmail.com 1234567890 Tirupati21 1 2025-03-01 150.50 Completed
2 Rani rani2766@gmail.com 2345678901 Tirupati22 2 2025-03-02 200.00 Pending
3 banu banu@gmail.com 3456789012 Bangalore 23 3 2025-03-03 99.99 Shipped
4 Charan charan@gmail.com 4567890123 chittoor 24 4 2025-03-04 49.95 Cancelled
5 surya surya427.com 5678901234 kurnool 25 5 2025-03-05 300.75 Completed
*/

--Query 17. Select customers whose names start with 'R'

```
SELECT * FROM CUSTOMER
WHERE Name LIKE 'R%';
```

OUTPUT:

/*1 Ravi ravi313@gmail.com 1234567890 Tirupati
2 Rani rani2766@gmail.com 2345678901 Tirupati
15 raji raji245@gmail.com 5555555555 kurnool
18 Roshini roshini34@gmail.com 8888888888 chennai*/

-- Query18. Select products whose names contain 'Lip'

```
SELECT * FROM PRODUCT
WHERE Name LIKE '%Lip%';
```

OUTPUT:

/*81 SuperStay Matte Ink Lipstick 141 121 9.99 150 2026-06-01
88 Hydrating Lip Balm 148 128 5.99 300 2026-10-30
97 Gloss Bomb Universal Lip Luminizer 157 137 24.99 120 2026-12-12*/

-- Query19. Select discounts that end in the last day of a month

```
SELECT * FROM DISCOUNT
```

WHERE EndDate LIKE '%-31';

OUTPUT:

/*NO OUTPUT*/

-- Query20. Order orders by order date in ascending order

SELECT * FROM ORDER_TABLE

ORDER BY OrderDate ASC;

OUTPUT:

/*21	1	2025-03-01	150.50	Completed
22	2	2025-03-02	200.00	Pending
23	3	2025-03-03	99.99	Shipped
24	4	2025-03-04	49.95	Cancelled
25	5	2025-03-05	300.75	Completed*/

-- Query21. Join ORDER_TABLE and ORDERDETAILS tables

SELECT * FROM ORDER_TABLE

JOIN ORDERDETAILS ON ORDER_TABLE.OrderID = ORDERDETAILS.OrderID;

OUTPUT:

/*21	1	2025-03-01	150.50	Completed	161	21	81	2	19.98
22	2	2025-03-02	200.00	Pending	162	22	82	1	34.99
23	3	2025-03-03	99.99	Shipped	163	23	83	3	32.97
24	4	2025-03-04	49.95	Cancelled	164	24	84	1	49.99
25	5	2025-03-05	300.75	Completed	165	25	85	2	59.98*/

--Query 22. Order customers by name alphabetically

SELECT CustomerID, Name, Email, Phone, Address

FROM CUSTOMER

ORDER BY Name ASC;

OUTPUT:

/*9	Balu	balu2867@gmail.com	9012345678	nellore
3 banu	banu@gmail.com	3456789012	Bangalore	
4 Charan	charan@gmail.com	4567890123	chittoor	
8 Devi	devi2786@gmail.com	8901234567	chennai	
6 janu	janu527@gmail.com	6789012345	Hyderabad*/	

-- Query23. Order payments by payment date in descending order

SELECT PaymentID, OrderID, PaymentStatus, PaymentMethod, TransactionID

FROM PAYMENT

ORDER BY PaymentID DESC;

OUTPUT:

/*60 40 Success Credit Card TXN1020
59 39 Success Debit Card TXN1019
58 38 Success PayPal TXN1018
57 37 Failed Credit Card TXN1017
56 36 Success Debit Card TXN1016

-- Query24. Get the average order total

```
SELECT AVG(TotalAmount)
FROM ORDER_TABLE;
```

OUTPUT:

/*182.293500*/

-- Query25. Get the total revenue from sales

```
SELECT SUM(TotalRevenue)
FROM SALES;
```

OUTPUT:

/*5782.83*/

-- Query26. Get the total amount spent by each customer

```
SELECT CustomerID, SUM(TotalAmount)
FROM ORDER_TABLE
GROUP BY CustomerID;
```

OUTPUT:

/*1 150.50
2 200.00
3 99.99
4 49.95
5 300.75
6 175.20
7 250.00
8 120.50
9 75.30
10 190.00
11 89.99
12 210.75
13 330.40
14 99.90
15 410.60

16 115.50
17 60.75
18 299.99
19 240.80
20 175.00*/

-- Query27. Get the number of orders with each status

```
SELECT Status, COUNT(*)
FROM ORDER_TABLE
GROUP BY Status;
```

OUTPUT:

/*Cancelled 2
Completed 8
Pending 4
Shipped 6*/

--Query 28. Get products that will expire in 2026

```
SELECT ProductID, Name, ExpiryDate
FROM PRODUCT
WHERE ExpiryDate LIKE '2026%';
```

OUTPUT:

/*81 SuperStay Matte Ink Lipstick 2026-06-01
82 Studio Fix Fluid Foundation 2026-08-15
83 Voluminous Lash Paradise Mascara 2026-05-10
85 Radiant Creamy Concealer 2026-09-20
87 Gel Envy Nail Polish 2026-07-12
88 Hydrating Lip Balm 2026-10-30
89 Dream BB Fresh Cream 2026-11-15
90 Pore Minimizing Primer 2026-12-01
91 All Nighter Setting Spray 2026-08-05
93 Matte Bronzer 2026-06-25
94 Tattoo Liner 2026-05-22
95 Brow Wiz Eyebrow Pencil 2026-09-30
96 Loose Setting Powder 2026-07-08
97 Gloss Bomb Universal Lip Luminizer 2026-12-12
99 Beauty Blender Sponge 2026-04-15*/

-- Query29. Get the names of customers who have placed orders

```
SELECT DISTINCT c.Name
```

```
FROM CUSTOMER c
JOIN ORDER_TABLE o ON c.CustomerID = o.CustomerID;
```

OUTPUT:

```
/*Balu
banu
Charan
Devi
janu
krishna
meghana
nandini
pavani
prakash
raji
Rani
Ravi
Roshini
santhosh
surya
teja
thanuja
veena
vinitha*/
```

-- Query30. Get the order IDs and payment methods for successful payments

```
SELECT o.OrderID, p.PaymentMethod FROM ORDER_TABLE o
JOIN PAYMENT p ON o.OrderID = p.OrderID
WHERE p.PaymentStatus = 'Success';
```

OUTPUT:

```
/*21      Credit Card
23      Debit Card
25      PayPal
26      Credit Card
28      PayPal
29      Credit Card
30      Debit Card
32      Credit Card
33      Debit Card
35      Credit Card
```

36 Debit Card
38 PayPal
39 Debit Card
40 Credit Card*/

--Query 31. Get the names of products reviewed with a rating of 5

```
SELECT p.Name FROM PRODUCT p
JOIN REVIEW r ON p.ProductID = r.ProductID
WHERE r.Rating = 5;
```

--Query 32. Get the names of suppliers who have supplied products

```
SELECT DISTINCT s.SupplierName
FROM SUPPLIER s
JOIN PURCHASE p ON s.SupplierID = p.SupplierID;
```

-- Query33. Get the order details for orders placed in March 2025

```
SELECT od.OrderDetailsID, od.OrderID, od.ProductID, od.Quantity, od.SubTotal
FROM ORDERDETAILS od
JOIN ORDER_TABLE o ON od.OrderID = o.OrderID
WHERE o.OrderDate LIKE '2025-03%';
```

--Query 34. Get the names of employees who are managers

```
SELECT Name FROM EMPLOYEE
WHERE Role LIKE '%Manager%';
```

-- Query35. Get the names of products from the brand 'Maybelline'

```
SELECT p.Name FROM PRODUCT p
JOIN BRAND b ON p.BrandID = b.BrandID
WHERE b.BrandName = 'Maybelline';
```

-- Query36. Get the order IDs and total amounts for orders with a total amount greater than 200

```
SELECT OrderID, TotalAmount FROM ORDER_TABLE
WHERE TotalAmount > 200;
```

-- Query37. Get the product names and prices for products in the 'Lipsticks' category

```
SELECT p.Name, p.Price FROM PRODUCT p
JOIN CATEGORY c ON p.CategoryID = c.CategoryID
WHERE c.CategoryName = 'Lipsticks';
```

-- Query38. Get the purchase IDs and total costs for purchases made after '2025-06-01'

```
SELECT PurchaseID, TotalCost FROM PURCHASE
WHERE PurchaseDate > '2025-06-01';
```

-- Query39. Get the average rating for each product

```
SELECT ProductID, AVG(Rating) FROM REVIEW
GROUP BY ProductID;
```

-- Query40. Get the number of reviews for each customer

```
SELECT CustomerID, COUNT(*) FROM REVIEW
GROUP BY CustomerID;
```

-- Query41. Get the total quantity of each product sold

```
SELECT ProductID, SUM(Quantity) FROM ORDERDETAILS
GROUP BY ProductID;
```

-- Query42. Get the number of purchases from each supplier

```
SELECT SupplierID, COUNT(*) FROM PURCHASE
GROUP BY SupplierID;
```

--Query 43. Get the names of customers who have not placed any orders

```
SELECT Name FROM CUSTOMER
WHERE CustomerID NOT IN (SELECT DISTINCT CustomerID FROM ORDER_TABLE);
```

--Query 44. Get the names of suppliers who have made any purchases

```
SELECT SupplierName FROM SUPPLIER
WHERE SupplierID IN (SELECT DISTINCT SupplierID FROM PURCHASE);
```

-- Query45. Get the order IDs and customer names for orders with status 'Completed'

```
SELECT o.OrderID, c.Name FROM ORDER_TABLE o
JOIN CUSTOMER c ON o.CustomerID = c.CustomerID WHERE o.Status = 'Completed';
```

-- Query46. Get the payment IDs and order IDs for payments made by 'PayPal'

```
SELECT PaymentID, OrderID FROM PAYMENT
WHERE PaymentMethod = 'PayPal';
```

-- Query47. Get the review IDs and customer names for reviews with rating 4 or 5

```
SELECT r.ReviewID, c.Name FROM REVIEW r
JOIN CUSTOMER c ON r.CustomerID = c.CustomerID WHERE r.Rating >= 4;
```

-- Query48. Get the discount names and discount percentages for discounts that start in '2025-03'

```
SELECT DiscountName, DiscountPercentage FROM DISCOUNT
WHERE StartDate LIKE '2025-03%';
```

-- Query49. Get the product names and category names for products with price above 30

```
SELECT p.Name, c.CategoryName FROM PRODUCT p
JOIN CATEGORY c ON p.CategoryID = c.CategoryID WHERE p.Price > 30;
```

-- Query50. Get the order details IDs and product names for order ID 25

```
SELECT od.OrderDetailsID, p.Name FROM ORDERDETAILS od
JOIN PRODUCT p ON od.ProductID = p.ProductID WHERE od.OrderID = 25;
```

-- Query51. Get the purchase IDs and supplier names for purchases with total cost above 1000

```
SELECT p.PurchaseID, s.SupplierName FROM PURCHASE p
JOIN SUPPLIER s ON p.SupplierID = s.SupplierID WHERE p.TotalCost > 1000;
```

-- Query52. Get the sales IDs and employee names for sales made on '2024-03-05'

```
SELECT s.SalesID, e.Name FROM SALES s
JOIN ORDER_TABLE o ON s.OrderID = o.OrderID
 JOIN CUSTOMER c ON o.CustomerID = c.CustomerID
JOIN EMPLOYEE e ON c.CustomerID = e.EmployeeID WHERE s.SalesDate = '2024-03-05';
```

-- Query53. Get the inventory IDs and product names for products with reorder level above 40

```
SELECT i.InventoryID, p.Name FROM INVENTORY i
JOIN PRODUCT p ON i.ProductID = p.ProductID
WHERE i.ReorderLevel > 40;
```

-- Query54. Get the average price of products in each category

```
SELECT c.CategoryName, AVG(p.Price) FROM PRODUCT p
JOIN CATEGORY c ON p.CategoryID = c.CategoryID
GROUP BY c.CategoryName;
```

-- Query55. Get the average discount percentage.

```
SELECT AVG(DiscountPercentage) FROM DISCOUNT;
```

-- Query56. Get the average total amount of orders placed by customer ID 1.

```
SELECT AVG(TotalAmount) FROM ORDER_TABLE
WHERE CustomerID = 1;
```

-- Query57. Get the average rating of reviews for product ID 81.

```
SELECT AVG(Rating) FROM REVIEW
WHERE ProductID = 81;
```

-- Query 58. Get the average cost price of products purchased from supplier ID 181.

```
SELECT AVG(CostPrice) FROM PURCHASEDETAILS
WHERE PurchaseID IN (SELECT PurchaseID FROM PURCHASE WHERE SupplierID = 181);
```

-- Query59. Get the average price of products from each brand.

```
SELECT b.BrandName, AVG(p.Price) FROM PRODUCT p
JOIN BRAND b ON p.BrandID = b.BrandID
GROUP BY b.BrandName;
```

-- Query60. Get the average cost price of products purchased from each supplier.

```
SELECT s.SupplierName, AVG(pd.CostPrice) FROM PURCHASEDETAILS pd
JOIN PURCHASE p ON pd.PurchaseID = p.PurchaseID JOIN SUPPLIER s ON p.SupplierID =
s.SupplierID
GROUP BY s.SupplierName;
```

-- Query61. Get the average rating of reviews with comments containing 'Amazing'.

```
SELECT AVG(Rating) FROM REVIEW
WHERE Comment LIKE '%Amazing%';
```

-- Query62. Get the average total revenue of sales made by employees with salary below 35000.

```
SELECT AVG(TotalRevenue) FROM SALES
WHERE OrderID IN (SELECT OrderID FROM ORDER_TABLE WHERE CustomerID IN (SELECT
CustomerID FROM EMPLOYEE WHERE Salary < 35000));
```

-- Query63. Get the average price of products that are not in the inventory.

```
SELECT AVG(Price) FROM PRODUCT
WHERE ProductID NOT IN (SELECT ProductID FROM INVENTORY);
```

-- Query64. Get the average total amount of orders that have not been paid.

```
SELECT AVG(TotalAmount) FROM ORDER_TABLE
WHERE OrderID NOT IN (SELECT OrderID FROM PAYMENT);
```

-- Query65. Get the average rating of reviews that have no comments.

```
SELECT AVG(Rating) FROM REVIEW
WHERE Comment IS NULL;
```

-- Query66. Get the average total revenue of sales made by employees who have not sold any products.

```
SELECT AVG(TotalRevenue) FROM SALES
WHERE OrderID IN (SELECT OrderID FROM ORDER_TABLE
WHERE CustomerID IN (SELECT CustomerID FROM EMPLOYEE
WHERE EmployeeID NOT IN (SELECT c.CustomerID FROM ORDER_TABLE o JOIN CUSTOMER
c ON o.CustomerID = c.CustomerID)));
```

--Query 67. Find the customer who placed the order with the highest total amount.

```
SELECT c.Name
FROM CUSTOMER c
JOIN ORDER_TABLE o ON c.CustomerID = o.CustomerID
WHERE o.TotalAmount = (SELECT MAX(TotalAmount) FROM ORDER_TABLE);
```

-- Query68. List all customers who have placed orders but have not made any payments.

```
SELECT c.Name
FROM CUSTOMER c
JOIN ORDER_TABLE o ON c.CustomerID = o.CustomerID
WHERE o.OrderID NOT IN (SELECT OrderID FROM PAYMENT);
```

-- Query69. List all brands that have products with stock quantity below 50.

```
SELECT b.BrandName
FROM BRAND b
JOIN PRODUCT p ON b.BrandID = p.BrandID
JOIN INVENTORY i ON p.ProductID = i.ProductID
WHERE i.StockLevel < 50;
```

--Query 70. List all orders that have associated payments but have not been marked as 'Completed'.

```
SELECT o.OrderID
FROM ORDER_TABLE o
JOIN PAYMENT p ON o.OrderID = p.OrderID
WHERE o.Status != 'Completed';
```

-- Query71. Find products that have a price higher than the average price of their category.

```
SELECT p.Name, p.Price, cat.CategoryName
FROM PRODUCT p
```

```
JOIN CATEGORY cat ON p.CategoryID = cat.CategoryID
WHERE p.Price > (SELECT AVG(Price) FROM PRODUCT WHERE CategoryID = p.CategoryID);
```

--Query73. Find max price in category

```
SELECT CategoryID, MAX(Price) AS MaxPrice
FROM PRODUCT
GROUP BY CategoryID;
```

--Query74. Monthly Order Count

```
SELECT MONTH(OrderDate) AS Month, COUNT(OrderID) AS OrderCount
FROM ORDER_TABLE
GROUP BY MONTH(OrderDate);
```

--Query75. Employees Above Avg Salary

```
SELECT * FROM EMPLOYEE
WHERE Salary > (SELECT AVG(Salary) FROM EMPLOYEE);
```

--Query76. Running Total By Order Date

```
SELECT OrderDate, SUM(TotalAmount) OVER (ORDER BY OrderDate) AS RunningTotal
FROM ORDER_TABLE;
```

--Query77. Unique Products Sold Count

```
SELECT COUNT(DISTINCT ProductID) AS UniqueProductsSold
FROM ORDERDETAILS;
```

--Query78. Unique Products Sold

```
SELECT CustomerID, TotalAmount,
    CASE WHEN TotalAmount > 1000 THEN TotalAmount * 0.9 ELSE TotalAmount END AS
DiscountedAmount
FROM ORDER_TABLE;
```

--Query79. Pending Order Customers

```
SELECT DISTINCT CustomerID
FROM ORDER_TABLE
WHERE Status = 'Pending';
```

--Query80. Product Stock Levels

```
SELECT P.Name, I.StockLevel, I.ReorderLevel
FROM INVENTORY I
```

JOIN PRODUCT P ON I.ProductID = P.ProductID;

--Query 81. Update the price of product with ID 101 to $25.

UPDATE PRODUCT SET Price = 25 WHERE ProductID = 101;

--Query 82. Change the address of customer with ID 1 to 'New Address'.

UPDATE CUSTOMER SET Address = 'New Address' WHERE CustomerID = 1;

-- Query83. Change the status of order with ID 21 to 'Shipped'.

UPDATE ORDER_TABLE SET Status = 'Shipped' WHERE OrderID = 21;

-- Query84. Delete the product with ID 101.

DELETE FROM PRODUCT WHERE ProductID = 101;

--Query 85. Count the number of products in the PRODUCT table.

SELECT COUNT(*) FROM PRODUCT;

--Query 86. Sum the total amount of all orders in the ORDER_TABLE.

SELECT SUM(TotalAmount) FROM ORDER_TABLE;

-- Query87. Delete all reviews with a rating of 1.

DELETE FROM REVIEW WHERE Rating = 1;

-- Query89. Sum the quantities of products purchased from suppliers with names containing 'Cosmetic'.

SELECT SUM(Quantity) FROM PURCHASEDETAILS
WHERE PurchaseID IN (SELECT PurchaseID FROM PURCHASE
WHERE SupplierID IN (SELECT SupplierID FROM SUPPLIER WHERE SupplierName LIKE '%Cosmetic%'));

-- Query90. List customer names and the order IDs they placed.

SELECT c.Name, o.OrderID
FROM CUSTOMER c
JOIN ORDER_TABLE o ON c.CustomerID = o.CustomerID;

-- Query 91. List order IDs and the payment methods used.

SELECT o.OrderID, p.PaymentMethod
FROM ORDER_TABLE o
JOIN PAYMENT p ON o.OrderID = p.OrderID;

-- Query 92. List review IDs and the names of the products reviewed.

```
SELECT r.ReviewID, p.Name
FROM REVIEW r
JOIN PRODUCT p ON r.ProductID = p.ProductID;
```

-- Query 93. List product names and their stock levels.

```
SELECT p.Name, i.StockLevel
FROM PRODUCT p
JOIN INVENTORY i ON p.ProductID = i.ProductID;
```

-- Query 94. List discount IDs and the end dates.

```
SELECT d.DiscountID, d.EndDate
FROM DISCOUNT d;
```

-- Query 95. List order IDs and the order dates.

```
SELECT o.OrderID, o.OrderDate
FROM ORDER_TABLE o;
```

-- Query 96. List all products that have never been ordered.

```
SELECT p.Name
FROM PRODUCT p
LEFT JOIN ORDERDETAILS od ON p.ProductID = od.ProductID
WHERE od.ProductID IS NULL;
```

-- Query 97. Update the price of products in the 'Lipsticks' category to be 10% higher.

```
UPDATE PRODUCT
SET Price = Price * 1.10
WHERE CategoryID = (SELECT CategoryID FROM CATEGORY WHERE CategoryName =
'Lipsticks');
```

-- Query 98. Delete all customers who have not placed any orders.

```
DELETE FROM CUSTOMER
WHERE CustomerID NOT IN (SELECT DISTINCT CustomerID FROM ORDER_TABLE);
```

-- Query 99. Show the total number of payments made by each payment method.

```
SELECT PaymentMethod, COUNT(*) AS PaymentCount
FROM PAYMENT
GROUP BY PaymentMethod;
```

-- Query 100. List discount names and percentages, only for discounts greater than 10%.

```
SELECT DiscountName, DiscountPercentage FROM DISCOUNT
WHERE DiscountPercentage > 10;
```

-- Query 101. List brand names and countries, only for brands from 'USA'.

```
SELECT BrandName, Country FROM BRAND WHERE Country = 'USA';
```

-- Query 102. List supplier names and contact names, only for suppliers with contact names starting with 'A'.

```
SELECT SupplierName, ContactName FROM SUPPLIER
WHERE ContactName LIKE 'A%';
```

-- Query 103. List payment IDs and transaction IDs, only for payments with transaction IDs starting with 'T'.

```
SELECT PaymentID, TransactionID FROM PAYMENT
WHERE TransactionID LIKE 'T%;
```

-- Query 104. List order detail IDs and product IDs, only for order detail IDs greater than 60.

```
SELECT OrderDetailsID, ProductID FROM ORDERDETAILS
WHERE OrderDetailsID > 60;
```

-- Query 105. List purchase IDs and supplier IDs, only for purchases from supplier ID 181.

```
SELECT PurchaseID, SupplierID FROM PURCHASE
WHERE SupplierID = 181;
```

-- Query 106. List discount IDs and discount percentages, only for discounts with percentages greater than 15.

```
SELECT DiscountID, DiscountPercentage FROM DISCOUNT
WHERE DiscountPercentage > 15;
```

-- Query 107. List order detail IDs and quantities, only for order details with quantities greater than 2.

```
SELECT OrderDetailsID, Quantity
 FROM ORDERDETAILS
WHERE Quantity > 2;
```

-- Query 108. Select the names of all products that have a stock level below 50.

```
SELECT Name
```

```
FROM PRODUCT
WHERE ProductID IN (SELECT ProductID FROM INVENTORY WHERE StockLevel < 50);
```

-- Query 109. Select the names and phone numbers of all customers whose phone numbers start with '9'.

```
SELECT Name, Phone
FROM CUSTOMER
WHERE Phone LIKE '9%';
```

-- Query 110. Select the payment IDs and transaction IDs of all payments with transaction IDs starting with 'T'.

```
SELECT PaymentID, TransactionID
FROM PAYMENT
WHERE TransactionID LIKE 'T%';
```

-- Query 111. Select the review IDs and customer IDs of all reviews written by customer ID 2.

```
SELECT ReviewID, CustomerID
FROM REVIEW
WHERE CustomerID = 2;
```

-- Query 112. Select the inventory IDs and product IDs of all inventory items with IDs greater than 280.

```
SELECT InventoryID, ProductID
FROM INVENTORY
WHERE InventoryID > 280;
```

-- Query 113. Select the names and addresses of all customers with addresses containing 'Tirupati'.

```
SELECT Name, Address
FROM CUSTOMER
WHERE Address LIKE '%Tirupati%';
```

-- Query 114. Select the payment IDs and order IDs of all payments with order IDs greater than 20.

```
SELECT PaymentID, OrderID
FROM PAYMENT
WHERE OrderID > 20;
```

-- Query 115. Select the names of customers who have placed orders.

```
SELECT Name FROM CUSTOMER
```

WHERE CustomerID IN (SELECT DISTINCT CustomerID FROM ORDER_TABLE);

-- Query 116. Select the order IDs of orders that have payments with 'Credit Card'.

SELECT OrderID FROM ORDER_TABLE

WHERE OrderID IN (SELECT OrderID FROM PAYMENT WHERE PaymentMethod = 'Credit Card');

-- Query 117. Select the review IDs of reviews for products with a rating of 5.

SELECT ReviewID FROM REVIEW

WHERE ProductID IN (SELECT ProductID FROM REVIEW WHERE Rating = 5);

-- Query 118. Select the names of customers who have placed orders for products supplied by 'Supplier A'.

SELECT Name

FROM CUSTOMER

WHERE CustomerID IN (SELECT CustomerID FROM ORDER_TABLE

WHERE OrderID IN (SELECT OrderID FROM ORDERDETAILS WHERE ProductID IN (SELECT ProductID FROM PURCHASEDETAILS WHERE PurchaseID IN (SELECT PurchaseID FROM PURCHASE WHERE SupplierID = (SELECT SupplierID FROM SUPPLIER WHERE SupplierName = 'Supplier A')))));

-- Query 119. Select the order IDs of orders that include products with stock levels below 50.

SELECT OrderID

FROM ORDER_TABLE

WHERE OrderID IN (SELECT OrderID FROM ORDERDETAILS

WHERE ProductID IN (SELECT ProductID FROM INVENTORY WHERE StockLevel < 50));

-- Query 120. Select all distinct brand names from the BRAND table.

SELECT DISTINCT BrandName FROM BRAND;

-- Query 121. Select the maximum price of any product.

SELECT MAX(Price) FROM PRODUCT;

-- Query 122. Select product names and the expiry dates.

SELECT p.Name, p.ExpiryDate

FROM PRODUCT p;

-- Query 123. Select category names and descriptions, only for categories with descriptions containing 'Makeup'.

SELECT CategoryName, Description FROM CATEGORY

```
WHERE Description LIKE '%Makeup%';
```

-- Query 124. Select discount names and start dates, only for discounts starting in 2024.

```
SELECT DiscountName, StartDate FROM DISCOUNT
WHERE StartDate LIKE '2024%';
```

-- Query 125. Select discount IDs and discount percentages, only for discounts with percentages greater than 15.

```
SELECT DiscountID, DiscountPercentage
FROM DISCOUNT WHERE DiscountPercentage > 15;
```

-- Query 126. Sum the stock levels of all products in the INVENTORY table.

```
SELECT SUM(StockLevel)
FROM INVENTORY;
```

-- Query 127. Count the distinct supplier IDs in the PURCHASE table.

```
SELECT COUNT(DISTINCT SupplierID)
FROM PURCHASE;
```

-- Query 128. Count the distinct category IDs that have products with prices above $30.

```
SELECT COUNT(DISTINCT CategoryID)
FROM PRODUCT
WHERE Price > 30;
```

-- Query 129. Count the distinct order IDs that have been reviewed.

```
SELECT COUNT(DISTINCT OrderID) FROM ORDERDETAILS
WHERE ProductID IN (SELECT ProductID FROM REVIEW);
```

-- Query 130. Sum the quantities of products purchased from supplier ID 181.

```
SELECT SUM(Quantity) FROM PURCHASEDETAILS
WHERE PurchaseID IN (SELECT PurchaseID FROM PURCHASE WHERE SupplierID = 181);
```

-- Query 131. Count the distinct customer IDs that have placed orders for products from the 'Maybelline' brand.

```
SELECT COUNT(DISTINCT CustomerID) FROM ORDER_TABLE
WHERE OrderID IN (SELECT OrderID FROM ORDERDETAILS
WHERE ProductID IN (SELECT ProductID FROM PRODUCT
WHERE BrandID = (SELECT BrandID FROM BRAND WHERE BrandName = 'Maybelline')));
```

-- Query 132. Sum the quantities of products purchased from suppliers with names containing 'Cosmetic'.

```
SELECT SUM(Quantity)
FROM PURCHASEDETAILS
WHERE PurchaseID IN (SELECT PurchaseID FROM PURCHASE WHERE SupplierID IN (SELECT SupplierID FROM SUPPLIER WHERE SupplierName LIKE '%Cosmetic%'));
```

-- Query 133. Count the distinct category IDs that have products that have been reviewed with comments containing 'Amazing'.

```
SELECT COUNT(DISTINCT CategoryID)
FROM PRODUCT
WHERE ProductID IN (SELECT ProductID FROM REVIEW WHERE Comment LIKE '%Amazing%');
```

-- Query 134. Count the distinct order IDs that have been placed by customers who have made payments with 'Credit Card' and have also left reviews with comments containing 'Good'.

```
SELECT COUNT(DISTINCT OrderID) FROM ORDER_TABLE
WHERE OrderID IN (SELECT OrderID FROM PAYMENT
WHERE PaymentMethod = 'Credit Card') AND OrderID IN (SELECT OrderID FROM ORDERDETAILS
WHERE ProductID IN (SELECT ProductID FROM REVIEW WHERE Comment LIKE '%Good%'));
```

-- Query 135. Sum the salaries of employees who have made sales to customers who have made payments with 'Credit Card'.

```
SELECT SUM(Salary) FROM EMPLOYEE
WHERE EmployeeID IN (SELECT EmployeeID FROM CUSTOMER WHERE CustomerID IN (SELECT CustomerID FROM ORDER_TABLE WHERE OrderID IN (SELECT OrderID FROM PAYMENT WHERE PaymentMethod = 'Credit Card')));
```

-- Query 136. INNER JOIN: Select product names and category names.

```
SELECT p.Name, c.CategoryName
FROM PRODUCT p
INNER JOIN CATEGORY c ON p.CategoryID = c.CategoryID;
```

-- Query 137. LEFT JOIN: Select customer names and order IDs, including customers who haven't placed orders.

```
SELECT c.Name, o.OrderID
FROM CUSTOMER c
LEFT JOIN ORDER_TABLE o ON c.CustomerID = o.CustomerID;
```

-- Query 138. RIGHT JOIN: Select order IDs and payment methods, including payments without corresponding orders.

```
SELECT o.OrderID, p.PaymentMethod
FROM ORDER_TABLE o
RIGHT JOIN PAYMENT p ON o.OrderID = p.OrderID;
```

-- Query 139. LEFT JOIN: Select customer names and review IDs, including customers who haven't written reviews.

```
SELECT c.Name, r.ReviewID
FROM CUSTOMER c
LEFT JOIN REVIEW r ON c.CustomerID = r.CustomerID;
```

-- Query 140. RIGHT JOIN: Select supplier names and purchase IDs, including suppliers who haven't made purchases.

```
SELECT s.SupplierName, pu.PurchaseID
FROM SUPPLIER s
RIGHT JOIN PURCHASE pu ON s.SupplierID = pu.SupplierID;
```

-- Query 141. LEFT JOIN: Select category names and product names, including categories without products.

```
SELECT c.CategoryName, p.Name
FROM CATEGORY c
LEFT JOIN PRODUCT p ON c.CategoryID = p.CategoryID;
```

-- Query 142. LEFT JOIN: Select product names and inventory IDs, including products not in inventory.

```
SELECT p.Name, i.InventoryID
FROM PRODUCT p
LEFT JOIN INVENTORY i ON p.ProductID = i.ProductID;
```

-- Query 143. RIGHT JOIN: Select supplier names and product names, including suppliers who haven't supplied any products.

```
SELECT s.SupplierName, p.Name
FROM SUPPLIER s
RIGHT JOIN PURCHASE pu ON s.SupplierID = pu.SupplierID
RIGHT JOIN PURCHASEDETAILS pd ON pu.PurchaseID = pd.PurchaseID
RIGHT JOIN PRODUCT p ON pd.ProductID = p.ProductID;
```

-- Query 144. LEFT JOIN: Select category names and average product prices, including categories

without products.

```
SELECT c.CategoryName, AVG(p.Price)
FROM CATEGORY c
LEFT JOIN PRODUCT p ON c.CategoryID = p.CategoryID
GROUP BY c.CategoryName;
```

-- Query 145. RIGHT JOIN: Select brand names and average product ratings, including brands without reviewed products.

```
SELECT b.BrandName, AVG(r.Rating)
FROM BRAND b
RIGHT JOIN PRODUCT p ON b.BrandID = p.BrandID
RIGHT JOIN REVIEW r ON p.ProductID = r.ProductID
GROUP BY b.BrandName;
```

-- Query 146. Select brand names that have products with stock below 50.

```
SELECT Brand Name FROM BRAND
WHERE Brand ID IN (SELECT BrandID FROM PRODUCT
WHERE ProductID IN (SELECT ProductID FROM INVENTORY WHERE StockLevel < 50));
```

-- Query 147. Select employee names who have made sales to customers who left reviews above rating 4.

```
SELECT Name FROM EMPLOYEE
WHERE EmployeeID IN (SELECT EmployeeID FROM CUSTOMER
WHERE CustomerID IN (SELECT CustomerID FROM ORDER_TABLE
WHERE OrderID IN (SELECT OrderID FROM ORDERDETAILS
WHERE ProductID IN (SELECT ProductID FROM REVIEW WHERE Rating > 4))));
```

-- Query 148. Select category names that have products reviewed by customers who paid with 'Credit Card'.

```
SELECT CategoryName FROM CATEGORY
WHERE CategoryID IN (SELECT CategoryID FROM PRODUCT
WHERE ProductID IN (SELECT ProductID FROM REVIEW
WHERE CustomerID IN (SELECT CustomerID FROM ORDER_TABLE
WHERE OrderID IN (SELECT OrderID FROM PAYMENT WHERE PaymentMethod = 'Credit Card'))));
```

-- Query 149. Group by CategoryID and order by average price in descending order.

```
SELECT CategoryID, AVG(Price) AS AveragePrice
FROM PRODUCT
```

```
GROUP BY CategoryID
ORDER BY AveragePrice DESC;
```

-- Query 150. Group by CategoryID and order by maximum price in descending order.

```
SELECT CategoryID, MAX(Price) AS MaxPrice
FROM PRODUCT
GROUP BY CategoryID
ORDER BY MaxPrice DESC
```

CHAPTER 4
CONCLUSION AND FUTUREWORK

4.1 Conclusion

We can conclude that the main benefits of an online cosmetics database include various features such as increased sales, reduced dependency on manual record-keeping, streamlined and standardized payments, improved inventory management, and enhanced customer insights. To achieve all these, a well-structured database is essential, and the requirements for the cosmetics database system have been detailed in previous chapters.

4.2 Future Work

Databases play a crucial role in storing, maintaining, and accessing structured information. For a cosmetics database, future enhancements could include:

- **Integration with an E-commerce Platform** – Developing a website or mobile app that allows global access to product catalogs, customer reviews, and personalized recommendations.
- **AI-Powered Personalization** – Implementing AI-driven analytics to suggest beauty products based on user preferences, skin type, and previous purchases.
- **Augmented Reality (AR) for Virtual Try-On** – Incorporating AR technology to enable customers to virtually try makeup products before purchasing.
- **Enhanced Security Measures** – Strengthening data protection protocols to ensure secure transactions and customer privacy.
- **Multi-Language and Multi-Currency Support** – Expanding accessibility to cater to international customers with diverse language and payment preferences.